

Knowva 커뮤니티 기능 정리

담당: 장윤성 / 역할: 부팀장, 커뮤니티 파트 구현

구분	내용
작성 목적	커뮤니티 게시판, 게시물 상세, 커뮤니티 프로필, 인기 게시물, 첨부 이미지, 공지사항 처리 흐름을 실제 구현 기준으로 정리했습니다.
기능 개요	사용자가 과목별 게시판에서 글을 작성하고 댓글, 좋아요, 스크랩 같은 반응을 남기며, 본인의 활동 내역을 커뮤니티 프로필에서 확인할 수 있도록 구성했습니다.
화면 자료	커뮤니티 메인 화면과 게시물 작성 화면 이미지를 추가해 실제 구현 화면의 구조를 함께 확인할 수 있게 했습니다.
핵심 포인트	게시글 조회수와 반응 수치를 분리하고, 새 글이 무조건 인기글이 되지 않도록 실제 커뮤니티처럼 조회수, 공감, 댓글 수를 기준으로 인기 게시글을 구성했습니다.

구분	내용
문서 구성	기능 개요, 커뮤니티 메인 화면, 게시물 작성 화면, Controller/Service 흐름, 프로필 구현, 데이터 처리 기준, 면접 Q/A 순서로 정리했습니다.

1. 커뮤니티 기능 개요

커뮤니티는 학습자가 과목별로 질문, 자유글, 공부 일지를 남기고 다른 사용자의 반응을 확인하는 공간입니다. 단순한 게시글 목록이 아니라 글 작성 이후 댓글, 공감, 스크랩, 신고, 프로필 집계까지 이어지는 흐름을 하나의 기능 단위로 구성했습니다.

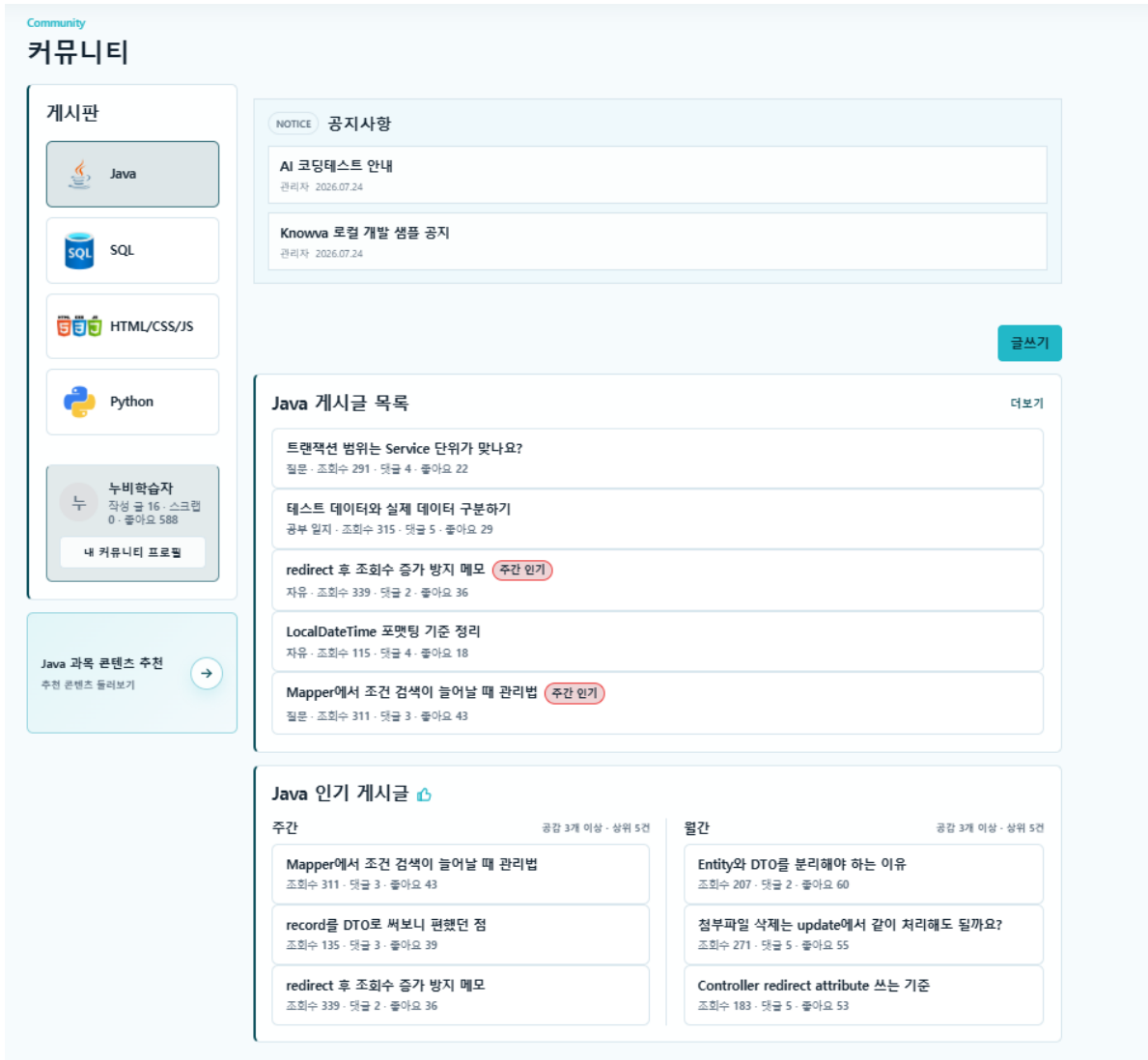
구분	내용
게시판 목록	Java, SQL, HTML/CSS/JS, Python 과목별 게시글을 보여주고 전체, 자유, 질문, 공부 일지 필터와 공감순, 조회수순 정렬을 제공합니다.
게시글 상세	게시글 제목, 작성자, 본문, 첨부 이미지, 조회수, 공감, 댓글, 스크랩, 신고 기능을 한 화면에서 처리합니다.
게시글 작성	제목, 언어, 게시판 유형, 본문 작성 영역을 제공하고 사용자가 커뮤니티 글을 등록할 수 있게 했습니다.
커뮤니티 프로필	작성 글, 작성 댓글, 받은 공감, 받은 댓글, 좋아요한 글, 스크랩한 글을 사용자 기준으로 집계합니다.
인기 게시글	주간/월간 기준으로 반응이 높은 게시글을 별도 대시보드에 노출해 사용자가 활발한 글을 빠르게 확인할 수 있게 했습니다.

구현 방향

- 목록 화면에서는 본문 내용을 숨기고 제목, 작성자, 댓글 수, 좋아요 수, 조회수를 중심으로 보여주었습니다.
- 상세 화면에서만 조회수가 증가하도록 처리해 좋아요, 댓글, 스크랩 같은 반응 동작과 조회수를 분리했습니다.
- 프로필 화면은 내가 쓴 글을 기본으로 보여주고, 세부 활동은 더보기 창으로 확인하게 하여 화면 복잡도를 줄였습니다.

2. 커뮤니티 메인 화면

커뮤니티 메인 화면은 왼쪽 과목 대시보드, 중앙 게시글 목록, 공지사항, 인기 게시글 영역을 한 화면에 배치했습니다. 사용자는 과목을 선택하고 공지사항을 확인한 뒤 게시글 목록과 인기 게시글을 자연스럽게 탐색할 수 있습니다.



구분	내용
공지사항	게시판 상단에 공지사항 영역을 두어 운영자가 전달해야 하는 내용을 먼저 확인할 수 있게 했습니다.
과목 대시보드	Java, SQL, HTML/CSS/JS, Python 과목을 아이콘과 함께 배치해 사용자가 원하는 과목 게시판으로 이동할 수 있게 했습니다.
게시글 목록	중앙에는 선택된 과목의 게시글을 표시하고 제목, 게시판 유형, 조회수, 댓글, 좋아요 수를 함께 보여줍니다.
인기 게시글	게시글 목록 하단에는 주간/월간 인기 게시글을 나누어 배치해 반응이 좋은 글을 빠르게 확인할 수 있게 했습니다.

3. 게시물 작성 화면

게시글 작성 화면은 사용자가 새 글을 작성할 때 필요한 필수 입력 흐름을 정리한 화면입니다. 제목, 언어, 게시판 유형, 본문 입력 영역을 분리하여 어떤 과목과 게시판에 글을 올리는지 명확하게 선택할 수 있게 했습니다.

구분	내용
제목 입력	게시글 제목을 먼저 입력하게 하여 목록과 상세 화면에서 핵심 내용이 잘 드러나도록 했습니다.
언어 선택	Java, SQL, HTML/CSS/JS, Python 중 작성 글이 속할 과목을 선택합니다.
게시판 선택	자유, 질문, 공부 일지 유형을 선택해 글의 목적을 구분할 수 있게 했습니다.
본문 작성	기본 모드와 Markdown 모드 탭을 두어 사용자가 글 작성 방식을 선택할 수 있게 구성했습니다.
등록/취소	작성 완료 시 등록 버튼으로 저장하고, 취소 버튼으로 이전 흐름으로 돌아갈 수 있게 했습니다.

4. Controller / Service 처리 흐름

구분	내용
게시글 목록	PostController / community/board.html / GET /community/board
게시글 상세	PostController / community/detail.html / GET /community/posts/{postId}
게시글 작성	PostController / community/write.html / POST /community/posts
게시글 수정	PostController / community/edit.html / POST /community/posts/{postId}/update
댓글 API	CommunityApiController / GET, POST, DELETE /api/community/posts/{postId}/comments
좋아요/스크랩	ReactionController 또는 CommunityApiController / POST /community/posts/{postId}/likes, scraps
프로필	CommunityProfileController / community/profile.html / GET /community/profile

```
@GetMapping("/community/board")
public String board(@RequestParam(required = false) Long subjectId,
    @RequestParam(defaultValue = "all") String boardType,
    @RequestParam(defaultValue = "popular") String sort,
    Model model, HttpSession session) {
    CommunityBoardView view = postService.board(subjectId, boardType, sort, session);
    model.addAttribute("view", view);
    return "community/board";
}
```

처리 기준

- Controller는 화면 조건을 받아 Service로 전달하고, Service는 게시글 목록, 인기글, 프로필 요약 데이터를 조합합니다.
- 댓글, 좋아요, 스크랩은 별도 요청으로 분리해 사용자의 반응만 갱신하도록 했습니다.
- 조회수는 상세 진입 시점에만 증가하도록 하여 버튼 클릭과 글 읽기 행위를 구분했습니다.

5. 커뮤니티 프로필 구현

커뮤니티 프로필은 사용자의 커뮤니티 활동을 정리해서 보여주는 화면입니다. 마이페이지와 역할이 겹치지 않도록 학습 프로필보다는 게시판 활동 중심으로 구성했습니다.

구분	내용
프로필 카드	닉네임, 프로필 이미지, 작성 글, 스크랩, 좋아요 수를 보여줍니다. 마이페이지에서 교체한 프로필 이미지는 원본 코드를 수정하지 않고 세션 값만 커뮤니티에 반영했습니다.
내가 쓴 글	프로필 진입 시 기본 영역으로 노출합니다. 사용자가 직접 작성한 글을 실제 커뮤니티 프로필처럼 제목 중심 카드로 확인할 수 있습니다.
작성 댓글	기본 화면에 바로 펼치지 않고 작성 댓글 제목 옆 더보기 버튼을 눌렀을 때 쪽지형 창에서 확인하게 했습니다.
받은 공감	내 글이 받은 공감 목록을 더보기 창으로 분리해 활동 데이터를 자세히 볼 수 있게 했습니다.
좋아요한 글/스크랩	사용자가 반응한 글 목록을 더보기 창으로 확인하게 하여 프로필 메인 화면이 과하게 길어지지 않게 했습니다.

설계 포인트

- 프로필 메인 화면은 작성 글 중심으로 구성해 사용자가 본인 활동을 직관적으로 파악할 수 있게 했습니다.
- 댓글, 좋아요, 스크랩처럼 목록이 길어질 수 있는 데이터는 더보기 창으로 분리했습니다.
- 마이페이지 담당 영역을 직접 수정하지 않고 커뮤니티에서 필요한 `profileImageUrl`만 읽어 사용하는 방식으로 범위를 지켰습니다.

6. 공지사항, 첨부 이미지, 조회수 처리

구분	내용
공지사항	데이터베이스에 있는 공지사항 내용을 읽어 과목별 게시판 상단에 노출합니다. 목록에서는 제목 중심으로 보이고 상세 진입 시 본문을 확인하게 했습니다.
첨부 이미지	게시글에 이미지를 첨부하면 상세 화면에서 첨부 파일 링크가 아니라 본문 영역 안에 이미지가 직접 보이도록 구성했습니다.
첨부 이미지 삭제	수정 화면에서 현재 첨부 이미지를 확인하고 삭제 체크를 통해 이미지를 제거할 수 있도록 했습니다.
조회수	상세 페이지 진입 시 증가하며, 좋아요/좋아요 취소, 댓글 등록/삭제, 스크랩 동작에서는 증가하지 않게 분리했습니다.
작성자 표시	게시글 작성 시 로그인 사용자의 실제 닉네임이 목록과 상세 화면에 반영되도록 했습니다.
인기글 기준	조회수, 좋아요, 댓글 반응이 높은 글을 기준으로 주간/월간 인기 게시글을 구성했습니다.

검증 기준

- 게시글 상세에 들어갔을 때만 조회수가 증가하는지 확인했습니다.
- 좋아요, 댓글, 스크랩 동작을 반복해도 조회수가 증가하지 않도록 분리했습니다.
- 첨부 이미지가 본문 안에 보이고, 수정 화면에서 삭제 체크박스과 삭제 문구가 한 줄로 정렬되도록 보완했습니다.

7. 면접 대비 Q/A

Q1. 커뮤니티 프로필을 별도 화면으로 만든 이유는 무엇인가요?

게시판 목록은 글을 탐색하는 화면이고 커뮤니티 프로필은 사용자의 활동을 확인하는 화면입니다. 목적이 다르기 때문에 분리했고, 작성 글, 작성 댓글, 받은 공감, 좋아요한 글, 스크랩한 글을 사용자 기준으로 정리할 수 있게 했습니다.

Q2. 새 글이 모두 인기글로 보이지 않게 한 이유는 무엇인가요?

실제 커뮤니티에서 인기글은 단순히 최신 글이 아니라 반응이 많은 글이어야 합니다. 그래서 조회수, 좋아요, 댓글 수를 기준으로 인기글을 구성하고 일반 게시글과 적절히 섞이도록 했습니다.

Q3. 게시글 작성 화면에서 언어와 게시판을 나눈 이유는 무엇인가요?

언어는 글이 속한 과목을 구분하고, 게시판은 글의 성격을 구분합니다. 두 값을 분리하면 Java 질문, Python 공부 일지처럼 검색과 필터링 기준을 명확하게 만들 수 있습니다.

Q4. 좋아요나 댓글을 눌렀을 때 조회수가 올라가지 않게 한 이유는 무엇인가요?

조회수는 사용자가 글을 읽기 위해 상세 화면에 진입한 횟수를 의미해야 합니다. 좋아요, 댓글, 스크랩은 반응 데이터이기 때문에 조회수와 분리해서 처리했습니다.

Q5. 다른 팀원의 범위를 침범하지 않기 위해 어떤 점을 신경 썼나요?

관리자, 로그인, 마이페이지 원본 기능은 직접 수정하지 않고 커뮤니티 화면에서 필요한 값만 읽어 사용하는 방식으로 작업했습니다. 기능 연결은 하되 담당 범위의 원본 코드는 유지하는 방향을 선택했습니다.